NexusHR

15-Day Implementation Plan — From Current State to Submission

> A solo-developer roadmap that hardens what's already built, closes the highest-risk gaps found in a full codebase audit, and builds real (not scoped-down) versions of the frontend, AI/RAG layer, and Kubernetes/EKS deployment — sequenced across 15 working days.

Project: NexusHR — AI-Enabled Enterprise HR & Workforce Intelligence Platform

Prepared: 2026-08-19 · **Demo due:** 2026-09-05

Scope basis: 3-agent codebase audit + roadmap design pass + dedicated AI/RAG & K8s/EKS design pass

Target: Zidio Development submission — PDF report, live demo, GitHub repo, README, demo video

Revision note: supersedes the earlier 10-day version — frontend, AI/RAG, and K8s/EKS restored to full build per request

Context

NexusHR is being built solo against an 8-week spec doc (dated March 2026) targeting a graded submission (100-pt rubric: Technical Depth 25, Functionality/UX 20, Docs 20, Innovation 15, Deployment 10, Presentation 10; submission checklist weights Live Demo 30%, Report 25%, GitHub 20%, README 15%, Video 10%). The demo is due **2026-09-05**.

A three-agent codebase audit plus two dedicated design passes verified exactly what's real versus aspirational in the spec. Headline findings:

- **Payroll batch pipeline** compiles but has **never been executed**. `PayrollProcessor` / `PayrollWriter` use `System.out.println` instead of SLF4J, and duplicate logic already in `PayrollServiceImpl.generateMonthlyPayroll()`. The batch `listener` / `scheduler` packages are empty.
- **Leave module** has dead code: `LeaveType.carryForwardAllowed` never read, half the `LeaveStatus` state machine never used, no `@Version` optimistic locking despite the spec claiming it.
- **Security gaps:** hardcoded plaintext JWT secret, no rate limiting, no general audit-log mechanism, no field-level PII encryption.
- **Testing:** one placeholder test. No Testcontainers, no API/E2E/load/security tooling.
- **Not started at all:** Performance Management, AI features, frontend (no `package.json` anywhere), Kubernetes/Helm/CI-CD, README, PDF report.
- **What's genuinely solid:** Auth (real Argon2id + Redis-backed refresh rotation), Employee/Department CRUD, Attendance with a real Redis pub/sub-backed SSE stream, Leave core flow, Payroll core (Draft→Approve→Lock→Paid), PDF payslips, migrations matching entities exactly.

> **Scope update (2026-08-19):** the first version of this plan scoped down the frontend, AI features, and Kubernetes/EKS deployment to fit a 10-day window. The user explicitly asked to restore those three to full builds, and extended the timeline to **15 days** to make that realistic. Testing depth and the smaller compliance/security extras (full 80%+ E2E coverage, field-level AES-256 PII encryption, SMS/Twilio, Form16/anomaly alerts) remain scoped down — those were not requested back.

Two Hard External Dependencies

- **OpenAI API key** (billed, small cost) — needed for RAG embeddings (`text-embedding-3-small`) + chat (`gpt-4o-mini`) on Day 9. Verify it works with a trivial call first thing that morning.
- **AWS account** with permissions to create an EKS cluster, ECR repos, and an IAM user — needed Day 13. Approx. cluster cost with the NAT-gateway-avoiding setup: **$5-8/day** — cheap enough to leave running continuously from Day 13 through the Sept 5 demo, then tear down right after.

Scope Decisions

🟢 Full build 🟣 Full build — restored this revision 🟠 Scoped down 🔴 Deferred

Area	Decision	Reason

Payroll batch hardening	**Full build**	80% done, cheap to finish, highest-risk "looks done but isn't" gap
PayrollAudit trail	**Full build**	Copy `LeaveAudit` pattern; financial mutations without audit is an obvious red flag
Leave `@Version` locking	**Full build**	One field + one migration, fixes a real bug
Leave accrual/carry-forward/state machine	**Full build**	Dead code actively hurts review if found
Async PDF/email	**Full build**	`@EnableAsync` + `@Async` on 2-3 methods
JWT secret + basic rate limiting	**Full build**	5-minute fix + a few hours; outsized security payoff
General AOP audit logging	**Scope down**	One `@Auditable` annotation + aspect + one table
Field-level AES-256 PII encryption	**Defer**	High effort, low marginal grading return
Performance Management	**Scope down**	Single-rater 1-5 review + goals, skip 360-multi-rater
AI attrition prediction	**Full build (restored)**	Real FastAPI sidecar + trained scikit-learn model — Days 6-7
RAG chatbot	**Full build (restored)**	Spring AI + PgVector + OpenAI over real HR policy docs — Day 9
Skill gap radar	**Defer**	Not part of the restore request; most cuttable Innovation item
Frontend	**Full build (restored)**	All modules incl. Performance/AI dashboards, dark mode, accessibility — Days 10-12
Kubernetes/EKS/Helm	**Full build (restored)**	Real `eksctl` cluster + Helm chart — Days 13-14. Trade-off: in-cluster Postgres/Redis instead of RDS/ElastiCache to fit the 2-day budget
CI/CD	**Scope down, still build**	`mvn test` + build on push
Prometheus/Grafana/Sentry	**Full build (restored)**	`kube-prometheus-stack` install + Sentry SaaS, backend-first — Day 14
JUnit/Mockito tests	**Full build**	Currently the single worst gap; highest ROI
Testcontainers	**Scope down to 3-5 tests**	Payroll batch e2e, leave approval+locking, auth login/refresh
Postman collection	**Scope down, still build**	Cheap, demoable
Playwright E2E	**Scope down to 3-5 journeys**	Not the full 80% target — not part of the restore request
k6 load test	**Scope down to one small run**	50-100 VUs, honestly framed as a smoke test
OWASP ZAP	**Scope down to one baseline scan**	Cheap, satisfies "security testing performed"
PF/ESI challan / Form16 / anomaly alerts	**Scope down to PF/ESI only**	Reuses `ReportController` pattern directly

Notification log / SMS	**Delivery log yes, SMS no**	Log table is cheap; Twilio is pure scope-add
README + PDF report	**Full build, started Day 3**	40% of submission checklist combined, currently zero

15-Day Schedule

Harden what's built (Days 1-3) → test infra + Performance Mgmt (Days 4-5) → real AI + RAG (Days 6-9) → full frontend (Days 10-12) → K8s/EKS + observability (Days 13-14) → docs/video/submission (Day 15). 15 days from 2026-08-19 lands on 2026-09-03, leaving a 2-day buffer before the 2026-09-05 demo.

Days 1-5 — Harden what's built, stand up test infra, finish Performance Mgmt

DAY 1 Payroll Batch: Make It Real

- Replace `System.out.println` with SLF4J/ `@Slf4j` in `PayrollProcessor.java` and `PayrollWriter.java`, mirroring `EmployeeLoggingProcessor.java`.
- Extract the dedupe→lookup→calculate→build sequence duplicated between `PayrollServiceImpl.generateMonthlyPayroll()` and `PayrollProcessor.process()` into one shared collaborator both call.
- Actually run `payrollJob` end-to-end against seeded local data; verify `payroll_records` rows and job execution status.

DAY 2 Payroll Batch: Operationalize It

- Add a `JobExecutionListener` in the empty `batch/listener` package; register on `payrollStep` / `payrollJob` in `BatchConfig.java`.
- Add a monthly `@Scheduled` trigger in the empty `batch/scheduler` package, styled on `LeaveScheduler.java`.
- Split test vs. real trigger: keep `BatchTestController` for the reader smoke test only; add `POST /api/payroll/batch/run?payrollMonth=` on `PayrollController`.
- Add `PayrollAudit` entity + `V21` migration (modeled on `LeaveAudit`); write rows on generate/approve/lock/pay.

DAY 3 Security + Leave Data-Integrity Fixes

- Add `@Version` to `LeaveBalance.java` + `V22` migration; verify optimistic-lock conflicts surface as 409s. *(Days 6-7 also add V21/V22-style AI migrations — renumber sequentially across workstreams.)*
- Externalize the JWT secret out of `application.properties` into an env var, no committed default.
- Add basic rate limiting (Bucket4j or a simple filter) on login/refresh only, alongside `JwtAuthenticationFilter` in `SecurityConfig.java`.
- Wire `LeaveType.carryForwardAllowed` and the full `LeaveStatus` state machine (`DRAFT` / `CANCELLED` / `CLOSED`) into real code.
- Start `README.md` — architecture, honest module-status table, local run instructions.

DAY 4 Async + Notifications + Test Infra Setup

- Add `@EnableAsync` / `@Async` to payslip PDF generation + email dispatch in `PayslipEmailServiceImpl.java` and `EmailServiceImpl.java`.
- Add a `notification_log` table + basic retry-once-on-failure.
- Add Testcontainers (`spring-boot-testcontainers` , `testcontainers-postgresql`) to `pom.xml` .
- Delete/replace the placeholder `AppTest.java` .

DAY 5 Test Suite + Performance Management + PF/ESI Challan

- 3-5 integration tests: payroll batch e2e, leave approve + `@Version` conflict, auth login/refresh rotation.
- Unit tests (JUnit5/Mockito) for `PayrollCalculator` , `PayrollServiceImpl` , the shared payroll helper, `LeaveScheduler` , `JwtUtils` .
- Scoped `@Auditable` AOP aspect + one `audit_log` table; apply to Employee/Department/Payroll mutations.
- Postman collection covering auth, employee, attendance, leave, payroll flows.
- `Goal` / `PerformanceReview` entities + migrations, standard repo/service/controller layers modeled on `DepartmentController` / `DepartmentService` .
- PF/ESI challan export, reusing the `ReportController.java` POI/iText pattern.
- Update README with testing instructions.

Days 6-9 — Real AI: Attrition Model + RAG Chatbot

Integration pattern: **Spring Boot computes features and POSTs them to FastAPI** (not FastAPI reading Postgres directly) — reuses existing repository logic, avoids DB credentials on the Python side, fits the existing chunk-oriented batch shape. Training data is a **synthetic, independent ~5,000-row dataset** — real historical attrition labels don't exist in this project, so a programmatic signal+noise generator is the honest way to get a trainable, >80%-accuracy-on-holdout model.

DAY 6 Data Foundation: Schema + Synthetic Dataset + FastAPI Skeleton

- Swap `docker-compose.yml` 's Postgres image to `pgvector/pgvector:pg17` ; recreate the local volume; `CREATE EXTENSION IF NOT EXISTS vector;` ; confirm all 20 existing Flyway migrations still apply.
- Add `V21__add_employee_ai_profile_fields.sql` : `hire_date` , `employment_status` , `exit_date` on `employees` ; backfill synthetic hire dates.
- Add `V22__create_ai_attrition_scores.sql` : `employee_id` FK, `risk_score` , `risk_band` , `scored_at` , `model_version` .
- Scaffold a top-level `ai-service/` FastAPI project (plain venv+pip): fastapi, uvicorn, scikit-learn, pandas, joblib, pydantic. Confirm `/health` locally on port 8000.
- Python synthetic-data generator: ~5,000 labeled rows across ~8-10 features (tenure, department, role level, overtime, absence rate, leave patterns, months since promotion, salary percentile) with injected signal + noise. *Time-box tuning to ~2 hours.*

DAY 7 Train the Model, Expose /predict, Wire the Java Client

- Train a `RandomForestClassifier` on the synthetic CSV; confirm >80% accuracy/AUC on holdout.
- Serialize the fitted pipeline as one `joblib` artifact.
- Build `POST /predict/batch` + single-item `/predict`; load the model once at startup.
- One-page feature-contract doc shared between Python training code and the Java DTO.
- Minimal multi-stage-ready `ai-service/Dockerfile` (python:3.12-slim, non-root user) now, to de-risk Day 13.
- Spring Boot: `RestClient` bean + `AttritionClient` service (~5s timeout); test in isolation via curl before wiring into a writer.

DAY 8 Spring Batch Job Wiring, Persistence, Scheduling

- `AttritionFeatureProcessor` (`@StepScope`) computes features via existing repositories, mirroring `PayrollProcessor` 's style.
- `AttritionScoreWriter` batches the whole chunk into one call to `AttritionClient.predictBatch()` ; wraps the call so a FastAPI outage logs+skips rather than failing the job.
- Register `attritionScoringStep` (chunk 20) and `attritionScoringJob` in `BatchConfig.java` , following the `payrollStep` / `payrollJob` shape.
- `AttritionScoringScheduler` (nightly, mirroring `LeaveScheduler.java`) + an admin trigger endpoint mirroring `BatchTestController.java` .
- End-to-end smoke test against seeded employees; spot-check directional sense of scores.

DAY 9 RAG Chatbot End-to-End

- **First thing in the morning:** verify the OpenAI key with a trivial embeddings call.
- Add Spring AI dependencies (BOM + openai-model-starter + pgvector-store-starter); configure `text-embedding-3-small` + `gpt-4o-mini` .
- Write 6-8 markdown HR policy docs (~400-800 words each): leave, attendance/overtime, payroll & benefits, code of conduct, remote work, performance review, exit/termination, grievance.
- Ingestion endpoint: `TokenTextSplitter` , embed, write into PgVectorStore; guard against duplicate re-ingestion.
- Chat endpoint via `ChatClient` + `QuestionAnswerAdvisor` (top-k=4); system prompt constrains answers to retrieved context; returns answer + source docs.
- Fallback: on OpenAI error/timeout, degrade to a plain SQL `LIKE` search tagged "degraded/no-AI" instead of a 500. Manually test 5-6 realistic questions.

Days 10-12 — Full React Frontend

DAY 10 Scaffold + Auth + Core CRUD

- Vite + React 19 + TS + TanStack Query + shadcn/ui + Tailwind scaffold.
- Auth: login page, JWT storage + refresh handling matching backend rotation, route guards by role.
- Employee list/detail/CRUD forms, Department hierarchy/org-chart view.
- Typed API client layer matching backend DTOs.

DAY 11 Attendance+SSE, Leave, Payroll, Performance

- Attendance board: check-in/out + a **live SSE panel** — most technically distinctive backend feature, currently zero visibility; prioritize over generic CRUD polish.
- Leave: apply form, balance display, manager approve/reject queue.
- Payroll: admin view by month, status transition buttons, payslip PDF download.
- Performance: goal list/create, review submission, review history.

DAY 12 AI Dashboards, Admin/Manager Dashboards, Polish

- AI Insights dashboard: ranked at-risk employee list (attrition scores + risk bands) + chatbot widget.
- Admin/Manager dashboard: department cost breakdown, live summary metric cards.
- Dark mode toggle, basic accessibility pass (keyboard nav, aria labels, contrast) — not a full WCAG AA audit.
- Mobile-responsive check on core screens.

Days 13-14 — Kubernetes/EKS + Observability

`eksctl`, 2-node `t3.medium`, public-subnet-only nodes (skip NAT gateways). **Postgres/Redis run in-cluster with PVCs, not RDS/ElastiCache** — a conscious, reversible spec deviation to fit the 2-day budget. One hand-written Helm chart. Prometheus+Grafana via one `kube-prometheus-stack` install; Sentry SaaS, backend-first. If Day 14 runs long, **Sentry is cut first**.

DAY 13 Containerize Everything, Stand Up EKS, Get Pods Running

- Multi-stage backend Dockerfile (Maven build → `eclipse-temurin:21-jre`). Multi-stage frontend Dockerfile (node build → nginx). ai-service Dockerfile already exists from Day 7.
- Push all three images to Amazon ECR.
- **First thing in the morning** (~15-20 min provisioning): `eksctl create cluster`, 2 nodes, `t3.medium`, no private networking; confirm nodes Ready.
- Author `charts/nexushr/`: Deployments+Services for backend/ai-service/frontend, Postgres+Redis with PVCs, one Secret, one `values.yaml`.
- `helm install`; iterate to all pods Running/Ready; expose frontend via LoadBalancer.
- *Risk mitigation:* prove the same images/env-vars work in docker-compose locally first, so K8s debugging is purely K8s mechanics.

DAY 14 Observability, Demo Rehearsal, Cost Discipline

- Add Actuator + micrometer-prometheus to backend, prometheus-fastapi-instrumentator to ai-service; expose metrics endpoints.
- `helm install kube-prometheus-stack` into its own `monitoring` namespace.
- ServiceMonitor for the backend; import one community JVM/Spring Boot Grafana dashboard.
- Sentry: sentry.io signup, backend SDK + DSN (minimum viable; sidecar/frontend SDKs only if time remains).
- Full end-to-end demo walkthrough against live EKS URLs; capture LoadBalancer/Grafana/Sentry links.
- Leave cluster running through Sept 5; `eksctl delete cluster` only after submission confirmed.

DAY 15 Report, Video, Submission

- Finalize README (env vars incl. `OPENAI_API_KEY`, architecture, known-limitations section — including the RDS/ElastiCache and skill-gap-radar deferrals, stated honestly).
- Write the PDF report (architecture, module-status table reusing this plan's scope-decision table, attrition model methodology + accuracy numbers, RAG design, test coverage, known limitations, screenshots).
- Playwright: 3-5 critical journeys against the live deployment. One small k6 run (50-100 VUs). One OWASP ZAP baseline scan.
- Record the 5-7 min demo video (SSE attendance, leave approval, payroll batch run + payslip, AI attrition dashboard, RAG chatbot).
- Repo cleanup: gate/remove `DebugController`, confirm no secrets committed, tag a release, verify submission checklist items.

Reuse Map — Do Not Rebuild These

- `PayrollServiceImpl.generateMonthlyPayroll()` — canonical dedupe/calc logic; `PayrollProcessor` should delegate, not duplicate.
- `ReportController.java` — POI/iText export template for PF/ESI challan and any future export.
- `LeaveAudit.java` — shape template for `PayrollAudit` and the generic `audit_log` table.
- `EmployeeLoggingProcessor.java` / `EmployeeLoggingWriter.java` — correct SLF4J pattern to copy into the payroll batch classes.
- `DepartmentController` / `DepartmentService` / `DepartmentServiceImpl` — cleanest multi-layer CRUD example; template for Performance Management.
- `LeaveScheduler.java` — `@Scheduled(cron=...)` pattern for the payroll and attrition-scoring triggers.
- `BatchConfig.java`'s `payrollStep` / `payrollJob` shape — template for the new `attritionScoringStep` / `attritionScoringJob`.
- `PayrollProcessor.java`'s repository-lookup style — template for `AttritionFeatureProcessor`'s feature computation.
- `BatchTestController.java` — template for the admin-trigger pattern (payroll, attrition scoring).
- `AttendanceSseController` / `AttendanceSseService` /Redis pub/sub chain — don't touch; foreground it in the frontend and demo video.
- `docker-compose.yml` / `Dockerfile` — extend (multi-stage, pgvector image) rather than replace.

Critical Files

- `Employee/src/main/java/org/Employee/batch/processor/PayrollProcessor.java`, `batch/writer/PayrollWriter.java`, `batch/config/BatchConfig.java` — payroll hardening + template for the attrition batch job.
- `Employee/src/main/java/org/Employee/service/PayrollServiceImpl.java`, `entity/LeaveBalance.java`, `service/scheduler/LeaveScheduler.java`, `service/PayslipEmailServiceImpl.java` — Days 1-5.
- `Employee/src/main/resources/application.properties`, `Employee/src/main/resources/db/migration/` (new V21+, watch for numbering collisions across workstreams).
- `Employee/src/main/java/org/Employee/controller/ReportController.java` — export pattern.
- `Employee/pom.xml` — Spring AI, pgvector-store, actuator+micrometer-prometheus, sentry-spring-boot-starter added across Days 6-9 and 14.
- `Employee/Dockerfile`, `Employee/docker-compose.yml` — extended across Days 6 (pgvector image) and 13 (multi-stage).
- New: `ai-service/` (FastAPI project, Days 6-7), `charts/nexushr/` (Helm chart, Day 13), frontend project root (Days 10-12).

Verification Checklist

- **Days 1-2:** `mvn -o compile` after each change; manually trigger `payrollJob` and confirm `payroll_records` / `payroll_audit` rows and job-execution logs.

- **Day 3:** confirm an optimistic-lock conflict is reproducible (two concurrent leave-balance updates → one gets a 409).
- **Days 4-5:** `mvn test` green with Testcontainers spinning up real Postgres; exercise the Postman collection; manual API check of Performance/PF-ESI endpoints via Swagger UI.
- **Day 7:** holdout accuracy/AUC >80% printed from the training script; `/predict` returns sane scores via Swagger UI.
- **Day 8:** `attritionScoringJob` run end-to-end locally populates `ai_attrition_scores` with directionally sensible values.
- **Day 9:** 5-6 manually-asked policy questions return grounded, cited answers; fallback path triggers correctly when the OpenAI key is temporarily invalidated.
- **Days 10-12:** `npm run dev` against the local backend; SSE panel updates live across two browser tabs; each module's CRUD flow exercised manually.
- **Day 13:** all pods `Running` / `Ready`; frontend LoadBalancer URL reachable.
- **Day 14:** Grafana dashboard shows live JVM metrics; Sentry receives a test event; full login→leave→payroll→AI flow works against the live EKS deployment.
- **Day 15:** Playwright green against the live deployment; k6 + ZAP reports captured; submission checklist items individually checked off.